DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-2

ASSIGNMENT 2 – CO2

CO2 Coverage: N-grams Models of Syntax; Counting Words; Unsmoothed N-grams; Smoothing — Backoff, Deleted Interpolation; Entropy; English Word Classes; Tagsets for English; Part of Speech Tagging; Rule-Based Part of Speech Tagging; Stochastic Part of Speech Tagging; Transformation-Based Tagging

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date: _____

Assignment Title: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline (Total Marks – 50)

Duration: 1 week

Scenario — Speech Transcript Analytics for a Transport Enquiry Helpline

A state transport corporation runs a telephone enquiry helpline that takes about four thousand calls a day about bus and train timings, routes, fares, cancellations and refunds. The calls are transcribed automatically, and the speech recogniser does not return one transcript but several competing hypotheses for every utterance: "two tickets to Vellore", "to tickets to Vellore" and "two tickets to Velore" all come back with similar acoustic scores. The recogniser cannot choose between them, because the decision is linguistic, not acoustic. Once a transcript is chosen it must be tagged before the origin, destination, date and fare can be pulled out of it, since "book", "return" and "stand" are each a noun in one call and a verb in the next.

You are the language-engineering team. You must build the n-gram language model that rescores the recogniser's hypotheses, measure it honestly with entropy and perplexity on held-out data, design a tagset for the domain, and build and compare three part-of-speech taggers
— rule-based, stochastic and transformation-based. Every claim you make about one model being better than another must rest on a measurement you made yourself.

Planning data and constraints

- Corpus: at least 20,000 transcribed utterances totalling at least 150,000 word tokens (real or synthetic, but documented), split 80 / 10 / 10 into training, development and test sets with no utterance appearing in more than one split.

- Recogniser output: at least 200 utterances for which five competing hypotheses and the correct reference transcript are available, so that rescoring can be evaluated.
- Tagset (design and document your own): between 20 and 30 tags covering the open and closed word classes of the domain, with an explicit mapping to the Penn Treebank tagset.
- Annotated data: at least 3,000 tokens hand-tagged with your tagset, of which at least 500 tokens are tagged independently by two people so that agreement can be reported.
- Evaluation: a held-out set that is never used for training or for tuning any weight, and a fixed random seed for every experiment so that results can be reproduced.
- Any manual effort (transcription, tagset design, annotation, rule writing, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The helpline is extended to metro and intercity rail services and begins to receive code-mixed speech, so the vocabulary grows sharply and more than fifteen per cent of the words in new calls are never seen in training. Re-evaluate your language model and your taggers and decide whether to retain, modify or redesign them.

TASK 1 – Counting Words and Building the Vocabulary 3 Marks
Guiding Enquiry

What exactly is being counted when we count the words of a corpus, and how do those decisions change every model built on top of them?

Investigate and interpret:

- Distinguish type, token, lemma and word form, and report all four counts for your corpus; state how you treated case, punctuation, numerals, disfluencies and the utterance boundary markers <s> and </s>.
- Justify every tokenisation decision that changes the counts — whether "Vellore" and "vellore" are one type, whether "6:30" is one token or three — and show the effect of two different decisions on the vocabulary size.
- Plot the frequency of each word against its rank, state Zipf's law, and check how closely your corpus follows it; identify the words that deviate most and explain why the domain causes the deviation.
- Build the vocabulary with an unknown-word threshold, mapping rare words to <UNK>; report the vocabulary size at thresholds of 1, 2, 3 and 5 and the proportion of tokens each threshold sends to <UNK>.
- Measure the out-of-vocabulary rate of the development set against the training vocabulary and explain why this single number limits everything that follows.

Evidence Expected

Documented counting decisions, a Zipf plot for the corpus, and vocabulary and OOV figures at several thresholds.

TASK 2 – Unsmoothed N-gram Models 4 Marks Guiding Enquiry

How is the probability of a word sequence estimated from counts alone, and where does that estimate break down?

Investigate and interpret:

- State the chain rule for the probability of a word sequence and derive the Markov assumption that turns it into a bigram and then a trigram model, saying precisely what is being assumed away at each step.
- Give the maximum likelihood estimator for unigram, bigram and trigram probabilities, and build all three models from your training split.
- Compute by hand the unsmoothed probability of at least three helpline utterances, showing every factor; choose one utterance that is perfectly reasonable English but receives probability zero, and identify the exact bigram responsible.
- Report the sparsity of your models: the proportion of possible bigrams and trigrams actually observed, and the proportion of development-set bigrams and trigrams never seen in training.
- Use the models as generators: sample ten utterances from the unigram, bigram and trigram models and comment on what the increase in order buys and what it costs.

Evidence Expected

Working unsmoothed models, hand-computed sentence probabilities including a zero, and quantified sparsity figures.

TASK 3 – Smoothing and Discounting 5 Marks Guiding
Enquiry
How can probability mass be moved from what was seen to what was not, without destroying what the counts tell us?

Investigate and interpret:

- Explain why a zero probability is a modelling failure rather than a fact about the language, and state the condition any smoothing method must satisfy for the distribution to remain valid.
- Implement add-one (Laplace) and add-k smoothing; for a bigram with a left context of your choice, tabulate the original counts, the effective (discounted) counts and the resulting probabilities, and show exactly how much mass was taken from the seen bigrams.
- Explain why add-one smoothing discounts so severely when the vocabulary is large, with the arithmetic for one context from your own corpus.
- Implement Good–Turing discounting: build the frequency-of-frequencies table N_r, compute the revised counts c*, estimate the probability of all unseen bigrams, and explain how the high-frequency tail is handled.
- Compare the methods on the development set by perplexity and by how reasonable the estimate for a specific unseen bigram looks, and state which you would deploy and why.

Evidence Expected

Correctly implemented smoothing methods with discount tables, a Good–Turing N_r table, and a measured comparison.

TASK 4 – Backoff and Deleted Interpolation 4 Marks
Guiding Enquiry
When the trigram has never been seen, how should the model fall back on the shorter contexts it does know?

Investigate and interpret:

- Define backoff and interpolation and state the essential difference: backoff consults a shorter context only when the longer one fails, interpolation always combines all orders.
- Implement Katz backoff with Good–Turing discounting, explaining the role of the normalising weight α and why the model is not a valid distribution without it; also implement stupid backoff and say why its output is a score and not a probability.
- Implement simple linear interpolation, then deleted interpolation: hold out part of the training data, estimate the λ weights on it, and report the weights you obtain for at least three context-count bands.
- Explain why the λ weights should depend on the count of the context, and show with one worked example from your corpus how a low-count context shifts weight onto the lower orders.
- Compare backoff and interpolation on the held-out set by perplexity, and hand-trace one utterance through both so that the difference in treatment of a single unseen trigram is visible.

Evidence Expected

Working backoff and interpolation models, estimated λ weights with the held-out procedure described, and a traced comparison.

TASK 5 – Entropy, Cross-Entropy and Perplexity 4 Marks Guiding Enquiry
How is a language model measured, and what does the number actually mean?

Investigate and interpret:

- Define entropy, cross-entropy and perplexity, state the relation between them, and explain in plain words what a perplexity of 90 tells an engineer about the model.
- Compute the entropy of at least three distributions from your own model by hand — including one that is nearly uniform and one that is sharply peaked — and explain what the difference means for prediction.
- Compute the cross-entropy and perplexity of the held-out set under each model built in Tasks 2 to 4, and present the results in one comparable table; state the per-word and per-utterance figures separately.
- Explain why perplexity may only be compared between models that share a vocabulary and the same treatment of <UNK>, and demonstrate the distortion with a deliberately unfair comparison of your own.
- Use your 200 recogniser hypothesis sets: rescore each set with your best model, report the word error rate before and after rescoring, and state whether perplexity improvements translated into error reductions.

Evidence Expected

Correct entropy calculations, a comparable perplexity table, and a rescoring experiment relating perplexity to word error rate.

TASK 6 – English Word Classes and Tagset Design 4 Marks Guiding
Enquiry
What are the word classes of English, and how many distinctions should a tagset for this helpline actually make?

Investigate and interpret:

- Distinguish open and closed word classes, list the members of the closed classes that occur in your corpus, and explain why the closed classes matter disproportionately for tagging.
- Define the major open classes with the morphological and distributional tests used to identify them, and apply those tests to at least five ambiguous words from your corpus.
- Design your tagset of 20 to 30 tags, justify each distinction you chose to make and at least three you chose not to make, and give the full mapping to the Penn Treebank tagset.
- Annotate your 3,000-token sample, report the tag frequency distribution, and give the inter-annotator agreement on the doubly annotated portion as a raw percentage and as Cohen's kappa.
- Measure the ambiguity your tagset leaves: the proportion of word types and of word tokens with more than one possible tag, and the average number of tags per ambiguous token.

Evidence Expected

A justified tagset with a Penn Treebank mapping, an annotated sample with agreement figures, and a measurement of residual ambiguity.

TASK 7 – Rule-Based Part-of-Speech Tagging 4 Marks Guiding
Enquiry
How far can hand-written rules take a tagger, and what makes writing them hard?

Investigate and interpret:

- Describe the two-stage architecture of a rule-based tagger: a dictionary that assigns every possible tag, then constraints that remove the impossible ones.
- Build the tag dictionary from your annotated data and report how many words receive one, two, three or more candidate tags.
- Write at least fifteen disambiguation rules in an explicit condition-action notation, each stating the context it inspects; include rules that use the preceding tag, the following word and the word's own suffix.
- Evaluate the tagger on the held-out set, report overall accuracy and accuracy on ambiguous tokens only, and explain why the second number is the one that matters.
- Analyse the failures: give the five commonest error types, show one case where two of your rules conflict, and explain how rule ordering resolved it and what that costs in maintainability.

Evidence Expected

A documented rule set with a tag dictionary, measured accuracy on ambiguous tokens, and a rule-conflict analysis.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm 5 Marks
Guiding Enquiry
How does a probabilistic tagger choose a whole tag sequence rather than one tag at a time?

Investigate and interpret:

- Derive the HMM tagging equation from Bayes' rule, state the two independence assumptions it rests on, and identify the transition and emission terms.
- Estimate the transition and emission probabilities from your annotated data, state how you smoothed them, and show that each transition row sums to one.
- Hand-trace the Viterbi algorithm on one ambiguous helpline utterance of at least five words: fill every trellis cell, keep a back-pointer for each, recover the best tag sequence and give its probability.
- Compare Viterbi against the most-frequent-tag baseline and against scoring every tag sequence exhaustively; report the accuracy of each and the number of operations the three require for your sentence.
- Explain how unknown words are tagged by an HMM — suffix evidence, capitalisation, an open-class prior — implement one such method, and report its accuracy on the out-of-vocabulary tokens alone.

Evidence Expected

A correctly estimated HMM, a fully traced Viterbi trellis with back-pointers, and a measured treatment of unknown words.

TASK 9 – Transformation-Based (Brill) Tagging 4 Marks Guiding
Enquiry
Can a tagger learn its own correction rules, and are the rules it learns worth reading?

Investigate and interpret:

- Describe transformation-based learning: the initial-state annotator, the templates, the greedy selection of the transformation that corrects the most errors, and the stopping condition.
- Define at least six rule templates over the preceding tag, the following tag, the surrounding words and the word's own form, and justify the window you chose.
- Run the learner from a most-frequent-tag initial state and report the first ten transformations learnt in order, with the number of errors each one corrected and introduced.
- Explain why the learnt rules are readable in a way that HMM probabilities are not, and show one learnt rule that encodes a genuine linguistic generalisation and one that is an artefact of your training data.
- Measure accuracy against the number of transformations applied, plot the curve, and identify the point beyond which further rules overfit.

Evidence Expected

A working Brill learner with its learnt rule list, and an accuracy curve identifying the overfitting point.

TASK 10 – Implement and Test the Models 3 Marks
Guiding Enquiry
Can a working implementation reproduce the behaviour you derived by hand?

Investigate and interpret:

- Implement in Python the counting and vocabulary pipeline, the unigram, bigram and trigram models with every smoothing method, backoff and interpolation, and all three taggers, using the standard library and NLTK only.
- Run every model over the same splits and report, for each: training time, model size in memory, held-out perplexity or tagging accuracy, and tagging speed in tokens per second.
- Include test cases for the zero-probability utterance from Task 2, the λ weights from Task 4, the trellis from Task 8 and the first transformations from Task 9, checking each against your hand-computed result.
- Present the results in comparable tables and at least two plots — for example perplexity against n-gram order, and tagging accuracy against training-set size.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs that match the hand calculations.

TASK 11 – Compare, Recommend and Defend the Design 2 Marks Guiding Enquiry
Which language model and which tagger would you deploy on the helpline, and what evidence supports the choice?

Investigate and interpret:

- Compare the smoothing and backoff strategies on perplexity, word error rate after rescoring, memory and speed, and compare the three taggers on accuracy, accuracy on ambiguous and unknown tokens, speed and ease of maintenance.
- Recommend one language model and one tagger, state the expected accuracy, the failure modes, and what a caller would actually experience when the system fails.
- Discuss the manual effort involved in annotation and rule writing, and identify the point at which more annotated data would help more than more rules.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest accuracy alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks
Changed Requirement

The helpline now also answers metro and intercity rail enquiries, so hundreds of new station names, train names and fare categories enter the vocabulary every month. Callers increasingly speak code-mixed sentences, for example "Chennai to Madurai bus timing enna", and the recogniser transcribes those words phonetically and inconsistently. More than fifteen per cent of the word tokens in new calls were never seen in training, many of them exactly the proper nouns that carry the caller's intent.

Re-evaluate your original design under the changed conditions. You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks
Guiding Enquiry
What exactly has changed, and which of your original assumptions no longer hold?

- Identify what the new input breaks: a closed vocabulary, a stable unknown-word rate, English-only word classes, transition probabilities estimated on a narrower domain, and a tagset that has no category for code-mixed tokens.
- Explain why the out-of-vocabulary words are the worst possible ones to lose, given that they are mostly the place names the enquiry turns on.
- Estimate the effect on perplexity and on tagging accuracy, reasoning from the OOV rate you measured in Task 1 and the unknown-word accuracy you measured in Task 8.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks
Guiding Enquiry
How would your recommended model and tagger behave on the new calls?

- Run or hand-trace at least three new utterances — one with an unseen station name, one code-mixed, one with both — through your language model and your chosen tagger, and record what fails first and why.
- Measure the degradation: held-out perplexity, word error rate after rescoring, and tagging accuracy on the new sample against your original baselines.
- Identify which component degrades fastest, and whether a different approach — a class-based or character-level model, an open-vocabulary tagger, a gazetteer of station names — would handle the new input more economically.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks
Guiding Enquiry
Should the models be retrained, extended or replaced?

- Choose retain, modify or redesign — for example adding a station-name gazetteer and a class-based n-gram over it, re-estimating λ weights on new data, extending the

tagset with a code-mixed category, or adding a character n-gram model for unseen words.

- Describe the revised design and state exactly what changes in the vocabulary, the smoothing, the backoff weights, the tagset and the tagger.
- Determine the effect on perplexity, tagging accuracy, retraining frequency and annotation effort, and say how often the model must now be refreshed.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your perplexity tables, rescoring results, tagging accuracies, error analyses and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what an n-gram model can never capture, however much data it is given.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

DSA03 – Natural Language Processing

Assignment 2: Build and Evaluate an N-gram Language Model and a Part-of-Speech Tagger for a Public Transport Enquiry Helpline

Introduction

The transport enquiry helpline receives questions about bus and train timings, routes, fares, cancellations and refunds. Speech recognition may produce several possible transcripts for one utterance. A language model can assign probabilities to these alternatives and help select the most likely sentence. After transcription, a part-of-speech (POS) tagger identifies the grammatical role of each word. This supports the extraction of information such as origin, destination, date and fare.

The proposed system uses unigram, bigram and trigram language models, smoothing and backoff methods, and three POS taggers: rule-based, Hidden Markov Model (HMM), and transformation-based (Brill) tagging.

Task 1 – Counting Words and Building the Vocabulary

A **token** is an individual occurrence of a word in the corpus. A **type** is a distinct word form. A **lemma** is the dictionary form of a word, while a **word form** is the actual form appearing in the sentence. For example, *book*, *books* and *booked* are different word forms but may share the lemma *book*.

For this project, text is converted to lowercase so that *Vellore* and *vellore* are treated as the same word. Punctuation is separated from words, numerals are retained, and disfluencies such as "uh" are kept because they occur in real speech. The sentence markers <s> and </s> are added to represent utterance boundaries.

Tokenisation decisions affect vocabulary size. For example, treating "6:30" as one token produces one type, while splitting it into "6", ":" and "30" produces three tokens. A consistent tokenisation method is necessary for training and evaluation.

Zipf's law states that a word's frequency is approximately inversely proportional to its rank:

$$f(r) \approx \frac{C}{r}$$

Very common words such as "to", "the" and "bus" occur frequently, while many station names and uncommon terms occur only a few times. The transport domain may cause deviations because words such as "ticket", "route" and station names occur more often than in general English.

Rare words can be replaced with <UNK> to reduce vocabulary size. A threshold of 1 keeps every word; thresholds of 2, 3 and 5 replace words occurring fewer times than the selected threshold. The OOV rate is:

$$\text{OOV rate} = \frac{\text{test tokens absent from training vocabulary}}{\text{total test tokens}} \times 100$$

The training vocabulary, OOV rate and vocabulary sizes at each threshold must be calculated from the actual corpus.

Task 2 – Unsmoothed N-gram Models

The chain rule expresses a sentence probability as:

$$P(w_1, \ldots, w_n) = \prod_{i=1}^{n} P\,(w_i, |, w_1 \ldots w_{i-1})$$

A unigram model assumes each word is independent of its context. A bigram model assumes that a word depends only on the previous word. A trigram model assumes that it depends only on the previous two words.

The maximum-likelihood estimates are:

$$P(w) = \frac{C(w)}{N}$$

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

$$P(w_i, \mid w_{i-2} w_{i-1}) = \frac{C(w_{i-2}, w_{i-1}, w_i)}{C(w_{i-2}, w_{i-1})}$$

For illustration, consider the small corpus:

- "book a ticket"
- "book a bus ticket"
- "a bus ticket"

The bigram probability of "ticket" after "bus" is:

$$P(\text{ticket} \mid \text{bus}) = \frac{C(\text{bus, ticket})}{C(\text{bus})} = \frac{2}{2} = 1$$

An unseen bigram receives probability zero. For example, if "bus to Vellore" never occurs in training, the unsmoothed bigram model assigns the whole sentence probability zero, even though it is a reasonable enquiry. This is a major weakness of unsmoothed models.

Higher-order models capture more context and can produce more fluent sequences, but they require more training data and have greater sparsity. The proportion of observed bigrams and trigrams, development-set unseen n-grams, and generated sample utterances must be reported from the actual corpus.

Task 3 – Smoothing and Discounting

A zero probability does not mean that a sentence is impossible; it often means only that the sentence was not observed in the training data. A valid smoothed model must assign non-negative probabilities whose total is 1.

Add-one (Laplace) smoothing is:

$$P(w_i \mid h) = \frac{C(h, w_i) + 1}{C(h) + V}$$

where $h$is the context and $V$is the vocabulary size.

Add-k smoothing uses a smaller constant k:

$$P(w_i \mid h) = \frac{C(h, w_i) + k}{C(h) + kV}$$

Add-one smoothing can over-discount frequent observed events when the vocabulary is large, because one count is added to every possible word. Add-k smoothing allows a smaller adjustment.

Good–Turing discounting estimates adjusted counts using the frequency-of-frequencies table N_r, where $N_r$is the number of n-grams occurring exactly $r$times:

$$c^* = (r+1)\frac{N_{r+1}}{N_r}$$

The probability mass assigned to unseen events is estimated using N_1/N, where $N_1$is the number of n-grams seen once and $N$is the total number of n-gram tokens. The actual count tables and resulting probabilities must be calculated from the training corpus.

Task 4 – Backoff and Deleted Interpolation

Katz backoff uses a higher-order n-gram probability when sufficient evidence exists. If the n-gram is unseen or discounted, the model backs off to a lower-order model and applies a backoff weight.

Stupid backoff uses a simple fixed multiplier when backing off to a lower-order n-gram. It is computationally efficient, but its scores are not guaranteed to form a probability distribution.

Linear interpolation combines unigram, bigram and trigram estimates:

$$P(w_i, \mid w_{i-2}w_{i-1}) = \lambda_1 P(w_i) + \lambda_2 P(w_i \mid w_{i-1}) + \lambda_3 P(w_i, \mid w_{i-2}w_{i-1})$$

The weights must satisfy:

$$\lambda_1 + \lambda_2 + \lambda_3 = 1,\ \lambda_i \geq 0$$

Deleted interpolation selects weights by testing how well each n-gram order predicts held-out events. The weights should be tuned on development data and then kept fixed for final test evaluation.

Task 5 – Entropy, Cross-Entropy and Perplexity

Cross-entropy measures how well a language model predicts a test sequence:

$$H(W) = -\frac{1}{N}\sum_{i=1}^{N} \log_2 P(w_i \mid context_i)$$

Perplexity is:

$$PP(W) = 2^{H(W)}$$

Lower cross-entropy and perplexity indicate that the model assigns higher probability to the held-out text. A zero probability causes infinite perplexity, which is another reason smoothing is required.

For speech-recognition rescoring, each hypothesis can be scored using its acoustic score and language-model score:

$$\text{Combined score} = \text{acoustic score} + \alpha \log P\,(W) + \beta \text{ word penalty}$$

The hypothesis with the best combined score is selected. Perplexity and word error rate (WER) must be evaluated on data not used for training or tuning.

Task 6 – English Word Classes and Tagset Design

The tagset should contain 20–30 tags and cover the words used in transport enquiries. A proposed 24-tag set is:

Tag	Meaning	Example
NOUN	Common noun	ticket
PROPN	Proper noun	Vellore
VERB	Main verb	travel
AUX	Auxiliary verb	can
ADJ	Adjective	express
ADV	Adverb	today
PRON	Pronoun	I
DET	Determiner	the
ADP	Preposition	to
CCONJ	Coordinating conjunction	and
SCONJ	Subordinating conjunction	because
NUM	Number	two
PART	Particle	to
INTJ	Interjection	hello

PUNCT	Punctuation	?
SYM	Symbol	₹
DATE	Date expression	Monday
TIME	Time expression	6:30
LOC	Location entity	Chennai
ROUTE	Route identifier	27A
FARE	Fare amount/category	₹50
TRANSPORT	Transport type	bus
DISFL	Disfluency	um
UNK	Unknown/unclassified token	unfamiliar form

The domain-specific tags can be mapped to Penn Treebank tags. For example, NOUN maps to NN/NNS, PROPN to NNP/NNPS, VERB to VB/VBD/VBG/VBN/VBP/VBZ, ADJ to JJ/JJR/JJS, ADV to RB/RBR/RBS, and PRON to PRP/PRP$. Domain tags such as LOC, ROUTE and FARE may map to NNP or CD while retaining their more specific labels for information extraction.

Annotation guidelines should define each tag and include examples of ambiguous words. Agreement between two annotators can be reported using percentage agreement or Cohen's kappa.

Task 7 – Rule-Based POS Tagging

A rule-based tagger assigns tags using word lists and grammatical rules. It can identify punctuation, numbers, known station names, common prepositions and common verbs. For example, a word following "to" may be a destination name, while a word ending in "-ing" may be a verb.

Rules are easy to interpret and maintain, but they require manual effort and may fail on ambiguous words. In "I want to book a ticket", *book* is a verb; in "the book is here", *book* is a noun. A rule-based tagger must use context to distinguish these cases.

The system should be evaluated on the same manually annotated test set as the other taggers. Report overall accuracy, accuracy on ambiguous words, accuracy on unknown words, speed and the main error types.

Task 8 – Stochastic HMM Tagging and the Viterbi Algorithm

An HMM POS tagger models the sequence of tags as hidden states and the words as observations. It uses transition probabilities between tags and emission probabilities from tags to words.

The best tag sequence is:

$$\hat{T} = \arg\max_{T} P(T, W)$$

The Viterbi algorithm calculates the best path efficiently:

$$V_i(t) = \max_{t'}[V_{i-1}(t')P(t \mid t')P(w_i \mid t)]$$

The algorithm stores the best score and previous tag for each position, then traces back from the final position to recover the tag sequence. Smoothing is needed for unseen transitions and emissions. Unknown words can be handled using suffix patterns, capitalization, word-shape features or an <UNK> emission.

The assignment's required trellis should be completed using probabilities estimated from the annotated training data. Tagging accuracy is calculated as:

$$\text{Accuracy} = \frac{\text{correctly tagged tokens}}{\text{total test tokens}} \times 100$$

Task 9 – Transformation-Based (Brill) Tagging

Brill tagging begins with an initial tag sequence, often produced by a simple tagger. It then learns correction rules from training data. A rule may change a tag when a specific context is present, for example, changing a noun tag to a verb tag when the previous word is “to”.

The learner repeatedly selects the rule that corrects the largest number of errors, subject to its scoring method. Rules are readable and can express linguistic patterns, but rules learned from a small corpus may overfit.

The first ten learned rules, the number of errors corrected and introduced, and accuracy after each rule must be recorded from the actual learner. The best stopping point is the point where development accuracy stops improving.

Task 10 – Implement and Test the Models

The implementation should use Python and NLTK, with a fixed random seed. The corpus should be divided by utterance into 80% training, 10% development and 10% test sets. No utterance should appear in more than one split.

The implementation should include:

1. Tokenisation, counting, vocabulary creation and <UNK> replacement.
2. Unigram, bigram and trigram models.
3. Add-one, add-k and Good–Turing smoothing.
4. Katz backoff, stupid backoff and interpolation.
5. Entropy, perplexity and hypothesis rescoring.
6. Rule-based, HMM and Brill POS taggers.
7. Tests comparing program outputs with hand calculations.

Report training time, model size, held-out perplexity, tagging accuracy and tagging speed. Include plots of perplexity against n-gram order and tagging accuracy against training-set size. These measurements must come from running the code; they should not be invented.

Task 11 – Compare, Recommend and Defend the Design

The final model should be selected using measured performance, memory, speed and maintenance effort—not accuracy alone. A smoothed trigram model with interpolation is a reasonable candidate because it uses local context while retaining lower-order estimates for sparse events. An HMM tagger is a reasonable candidate because it models tag sequence context and can be evaluated consistently.

The final recommendation must be confirmed by the actual test results. The report should discuss errors on ambiguous and unknown words, the effect of smoothing on perplexity and rescoring, and the effort required for annotation and rule writing. More annotated data is likely to be more useful than adding many rules when errors mainly involve ambiguous or unseen words.

Task 12 – Day-4 Design Challenge

The expanded service includes metro and intercity rail enquiries, many new station and train names, and code-mixed speech. More than 15% of new words are unseen in training. The original assumptions of a stable vocabulary and English-only input no longer hold.

OOV words are especially damaging because station names often identify the caller’s destination. If the model replaces a station name with <UNK>, it may fail to distinguish two competing transcripts with different destinations. A tagger may also assign a generic tag instead of identifying the location.

The original system should be **modified rather than retained unchanged**. The vocabulary should be updated with new station, train and fare names. A gazetteer can identify known locations, while character-level or subword features can help with unfamiliar spellings and phonetic variants. The tagset should include code-mixed and language-specific categories, and the tagger

should support unknown words.

The language model should be retrained or adapted using recent metro and intercity enquiries. Code-mixed examples should be included in training and annotation. The system should monitor OOV rate, perplexity, rescoring WER and tagging accuracy, and refresh the vocabulary and models when these measures degrade.

To evaluate the change, test at least three utterances: one containing an unseen station name, one code-mixed utterance, and one containing both. Compare their outputs and metrics against the original system. The expected direction is higher perplexity and lower tagging accuracy when OOV words increase, but the exact change must be measured on the new sample.

Conclusion

N-gram language models help rank speech-recognition hypotheses by estimating how likely a word sequence is. Smoothing and backoff reduce the problem of unseen n-grams, while POS tagging supports the extraction of useful information from transcripts. A reliable transport helpline system must be evaluated on held-out data and updated as its vocabulary and language use change.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Counting Words and Building the Vocabulary	3
Task 2 – Unsmoothed N-gram Models	4
Task 3 – Smoothing and Discounting	5
Task 4 – Backoff and Deleted Interpolation	4
Task 5 – Entropy, Cross-Entropy and Perplexity	4
Task 6 – English Word Classes and Tagset Design	4
Task 7 – Rule-Based Part-of-Speech Tagging	4
Task 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm	5
Task 9 – Transformation-Based (Brill) Tagging	4
Task 10 – Implement and Test the Models	3
Task 11 – Compare, Recommend and Defend the Design	2
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**